

FIGLUT: An Energy-Efficient Accelerator Design for FP-INT GEMM Using Look-Up Tables

Gunho Park^{1,2*}, Hyeokjun Kwon^{1*}, Jiwoo Kim¹, Jeongin Bae², Baeseong Park²,
Dongsoo Lee², and Youngjoo Lee¹

¹Pohang University of Science and Technology (POSTECH),

²NAVER Cloud

{gunho.park3}@navercorp.com, {kwon36hj, youngjoo.lee}@postech.ac.kr

Abstract—Weight-only quantization has emerged as a promising solution to the deployment challenges of large language models (LLMs). However, it necessitates FP-INT operations, which make implementation on general-purpose hardware like GPUs difficult. In this paper, we propose FIGLUT, an efficient look-up table (LUT)-based GEMM accelerator architecture. Instead of performing traditional arithmetic operations, FIGLUT retrieves precomputed values from an LUT based on weight patterns, significantly reducing the computational complexity. We also introduce a novel LUT design that addresses the limitations of conventional memory architectures. To further improve LUT-based operations, we propose a half-size LUT combined with a dedicated decoding and multiplexing unit. FIGLUT efficiently supports different bit precisions and quantization methods using a single fixed hardware configuration. For the same 3-bit weight precision, FIGLUT demonstrates 59% higher TOPS/W and 20% lower perplexity than state-of-the-art accelerator design. When targeting the same perplexity, FIGLUT achieves 98% higher TOPS/W by performing 2.4-bit operations.

I. INTRODUCTION

Pretrained Large Language Models (LLMs) have demonstrated remarkable performance in a wide range of language understanding and generation tasks. These models, predominantly based on the Transformer architecture [1], [3], [11], [29], [31], exhibit a predictable power-law scaling in performance relative to their parameter size. Consequently, the size of the model has increased to unprecedented levels, pushing the boundaries of what can be achieved in natural language processing. However, substantial growth in model size comes with significant challenges, particularly in terms of memory footprint. [26], [35] For instance, a model with 175 billion parameters, requiring approximately 350GB of memory, far exceeds the DRAM size of 80GB available in current state-of-the-art GPUs [28]. Additionally, memory bandwidth also becomes a critical issue, leading to significant memory bottlenecks. This bottleneck can severely impact the performance and efficiency of deploying such large models, as data transfer rates cannot keep up with the computational demands of the LLMs. This discrepancy highlights the urgent need for efficient methods to deploy such large models on hardware accelerator.

One of the most promising approaches to mitigate the memory-related challenges of LLM is parameter quantization [7], [15], [32]. In the context of LLMs, there has been

a shift toward weight-only quantization, where only weights are strongly quantized to sub-4-bit precision while activations remain in floating point (FP) format [4], [8]. This shift is motivated by the observation that the memory footprint of the weights significantly exceeds that of the activations in the LLM, and the presence of outliers in the activations further supports the effectiveness of weight-only quantization. Despite its advantages, weight-only quantization requires the use of non-standard FP-INT operations, posing an ongoing challenge.

Recent studies have introduced efficient kernels for weight-only quantization, enabling practical speed-ups on GPUs [10], [25], [28], [36]. Although the kernels proposed in [10], [25] demonstrated faster speeds compared to cuBLAS, which is a GPU-accelerated library to accelerate HPC applications, including GEMM operation, these improvements largely stem from efficient data movement due to weight compression. However, during computation, the compressed weights are dequantized back to FP format, resulting in inefficiencies as the actual arithmetic operations are still performed using FP-FP units. NVIDIA's recent release of new FP-INT GEMM kernels in CUTLASS [21] similarly relies on inherent FP operations. Efforts to overcome these inefficiencies have led to the proposal of LUT-based GEMM kernels [28], [36], which aim to perform FP-INT GEMM operations without dequantization. However, storing LUTs in shared memory on GPUs often leads to bank conflicts, causing inefficiencies because multiple threads access the same shared memory bank simultaneously.

To address these issues, recent advancements propose novel hardware architectures designed to efficiently perform FP-INT operations specifically tailored for weight-only quantized models [20]. These emerging solutions aim to bridge the gap between the theoretical benefits of weight-only quantization and practical deployment on existing hardware. For instance, iFPU [22] proposes an efficient bit-serial architecture accelerator utilizing binary-coding quantization (BCQ) format. In the BCQ format, the inner product of binary weights and input activations is replaced by the sum or subtraction of input activations. iFPU introduces a pre-alignment technique that aligns the exponents of input activations with FP values. Consequently, the operations on pre-aligned mantissas are handled by integer arithmetic units, efficiently performing FP-INT operations. Similarly, FIGNA [16] presents FP-

*Equal contribution.

TABLE I
COMPARISON OF DIFFERENT HARDWARE ACCELERATORS

	FP-INT Operation	Mixed- Precision	BCQ Support	Computational Complexity
GPU	✗	✗	✗	$\mathcal{O}(mnk)$
iFPU [22]	✓	✓	✓	$\mathcal{O}(mnkq)$
FIGNA [16]	✓	✗	✗	$\mathcal{O}(mnk)$
Proposed	✓	✓	✓	$\mathcal{O}(mnkq/\mu)$

INT4 multiplication employing the pre-alignment technique while addressing the inefficiencies inherent in the bit-serial structure adopted by iFPU. By using pre-alignment, FIGNA replaces FP-INT multiplication with integer multiplication between aligned mantissas and quantized weights, enhancing the efficiency of weight-only quantized model computations. However, FIGNA has limitations as it is fixed to a specific precision (e.g., 4-bit quantized weights), making it inefficient for models quantized to less than 4-bit precision or for those using mixed-precision quantization methods.

In this paper, we propose FIGLUT, a novel accelerator architecture that leverages LUT-based GEMM to reduce the computational complexity of bit-serial accelerators. FIGLUT replaces arithmetic operations with table look-ups using weight patterns as keys, significantly enhancing computational efficiency. We introduce a specialized operator, the read-accumulate (RAC) unit, which replaces the conventional multiply-accumulate (MAC) unit found in traditional hardware accelerators. FIGLUT efficiently supports FP-INT operations for weight-only quantized models and employs a bit-serial structure to manage computations of various precisions within a single hardware framework. Moreover, by using the BCQ format for weight representation, FIGLUT can operate not only on BCQ models, which present state-of-the-art performance, but also on commonly used uniformly quantized models. As a result, FIGLUT enables efficient computation across different bit precisions and quantization methods on a single hardware platform, representing a significant advancement in the deployment of weight-only quantized models.

Our major contributions in this work include the following:

- We propose an LUT-based FP-INT GEMM method that replaces arithmetic operations with LUT reads, facilitating energy-efficient GEMM operations.
- We introduce a novel specialized LUT architecture that enables simultaneous access to the LUT without bank conflicts during parallel processing.
- We present FIGLUT, an innovative LUT-based FP-INT GEMM accelerator that efficiently supports various quantization methods and precisions on a single hardware platform by leveraging LUT-based operations.

II. BACKGROUND

A. Methods and Systems for Weight-Only Quantization

Quantization is a powerful method for reducing computational complexity and memory footprint by converting high-bit precision FP values into low-bit INT values. However,

applying quantization to LLMs presents significant challenges, especially when attempting to quantize both weights and activations to lower bit precisions. Activation values, in particular, are more difficult to quantize than weight values due to the presence of outliers and their dynamic nature, which makes their distribution hard to predict accurately [9], [34]. This complexity necessitates specialized techniques to effectively manage the quantization of activations in LLMs.

To address memory-related challenges such as memory bound of LLM inference and limited DRAM size, weight-only quantization methods have been extensively studied. These methods preserve accuracy by keeping the activations in FP format. Due to the significant accuracy drop observed with simple round-to-nearest (RTN) quantization, various techniques have been proposed to minimize quantization error. OPTQ [10] introduces a uniform quantization method that utilizes approximate second-order information from a calibration dataset to achieve sub-4-bit quantization with negligible accuracy loss. AWQ [25] proposes a weight-only quantization approach that selects salient weights based on activation distribution to minimize quantization error. Recently, non-uniform quantization techniques using BCQ have also demonstrated excellent performance. ShiftAddLLM [36] achieves state-of-the-art results by applying post-training bitwise shift-and-add reparameterization with non-uniform BCQ, enabling a multiplication-less LLM. Additionally, ShiftAddLLM improves the accuracy-efficiency trade-off by employing mixed-precision quantization based on the sensitivity of each layer.

With the proven efficiency of weight-only quantization methods, various hardware accelerators have been proposed to support FP-INT operations between activations and weights. Table I compares the features of different hardware accelerators. Commercial GPUs lack FP-INT arithmetic units that can handle FP input activations and INT weights. As a result, a dequantization process is necessary to convert INT weights back to FP format for use with existing FP-FP arithmetic units. The FP-INT GEMM kernels proposed in [10] and [25] achieve better latency compared to cuBLAS FP-FP GEMM, thanks to efficient bandwidth utilization enabled by weight compression in memory-bound LLM applications. However, the absence of native FP-INT operations on GPUs undermines the advantages of weight quantization by necessitating FP-FP computations.

To address inefficiencies in GPUs and effectively handle FP-INT GEMM operations, several accelerator designs have been proposed. iFPU [22] introduces a method to efficiently process computation between weights in BCQ format and FP activations by pre-aligning the mantissas of input activations based on the maximum exponent value. This alignment allows FP-INT operations to be replaced by INT-INT addition between mantissas and binary weights, reducing hardware complexity. While this bit-serial approach enables support for mixed-precision quantization, it inherently increases computational complexity in proportion to the bit precision q of the quantized weights.

FIGNA [16] addresses the computational overhead of bit-serial architecture by proposing an FP-INT arithmetic unit

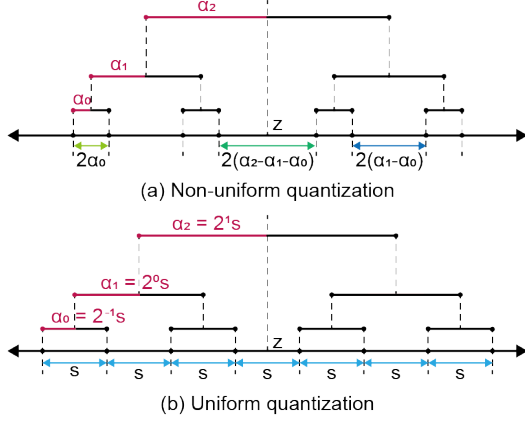


Fig. 1. Extension of binary-coding quantization to support both non-uniform and uniform quantization formats, achieved by including an offset term ($q = 3$).

that performs INT-INT multiplications between pre-aligned mantissa and uniformly quantized weights. While this approach effectively reduces computational complexity, it is constrained by its fixed precision, limiting its applicability to uniformly quantized models and precluding support for sub-4-bit precision or advanced BCQ-based non-uniform quantization methods. Consequently, there is a need for a new bit-serial hardware architecture that minimizes computational overhead while supporting a broader range of precisions and quantization methods, including non-uniform quantization with BCQ.

B. Binary Coding Quantization

Binary Coding Quantization (BCQ) [33] is a quantization technique that converts a real-valued weight value $w \in \mathbb{R}$ into q -bit precision representation. This is achieved by expressing w as a linear combination of binary weights $\{b_i\}_{i=1}^q$ and their corresponding scaling factors $\{\alpha_i\}_{i=1}^q$, where $b_i \in \{-1, 1\}$ [36]. The binary weights and scaling factors are optimized to minimize the quantization error using the following objective:

$$\arg \min_{\alpha_i, b_i} \left\| w - \sum_i \alpha_i b_i \right\|^2. \quad (1)$$

Due to the absence of an analytical solution for minimizing the quantization error, heuristic methods are typically employed to approximate the optimal scaling factors and binary matrices.

While BCQ can be used for both activation and weight quantization, it leads to higher computational complexity when applied to activation quantization [17]. Focusing on weight-only quantization to preserve accuracy, we will consider BCQ in the context of weight quantization. Consequently, the product of a FP activation vector $\mathbf{x} \in \mathbb{R}^{n \times 1}$ and a weight matrix quantized with binary matrices $\mathbf{B}_i \in \{-1, 1\}^{m \times n}$ and scaling factors $\alpha_i \in \mathbb{R}^{m \times 1}$ can be expressed as:

$$\mathbf{y} = \sum_{i=1}^q (\alpha_i \circ (\mathbf{B}_i \cdot \mathbf{x})) \quad (2)$$

where " $\circ$ " denotes the Hadamard product.

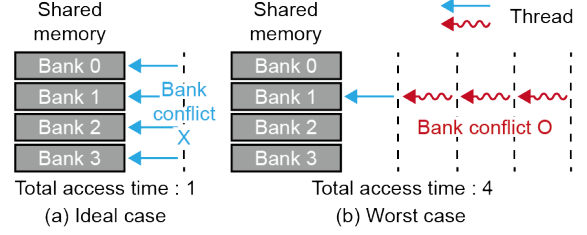


Fig. 2. Comparison of bank conflicts during shared memory access.

Conventional BCQ, a non-uniform quantization method, is incompatible with most state-of-the-art models that use uniform quantization. [28] enhanced BCQ by incorporating an offset term z , thereby improving its representational capacity:

$$w = \sum_{i=1}^q (\alpha_i \cdot b_i) + z \quad (3)$$

It has been demonstrated that uniform quantization can be effectively represented by appropriately adjusting the scaling factors and offset terms [28]. As shown in Fig. 1, conventional BCQ leverages multiple distinct scaling factors, while the addition of an offset term facilitates the representation of uniformly quantized values.

C. Bank Conflict

To perform efficient parallel operations, it is necessary to access multiple data simultaneously. Many parallel processors achieve this by employing memory banking techniques, which split memory into banks, allowing multiple threads to access the memory concurrently. However, memory banking can encounter issues when the data required in a single operation cycle are located in the same bank, leading to bank conflicts [12]. This problem is particularly evident in GPUs, where a shared memory space is utilized to enable efficient parallel operations among threads. When different threads attempt to access the same bank simultaneously, bank conflicts arise. These conflicts hinder the achievement of predicted performance and lead to computational resource wastage, as operations that could be parallelized are instead executed serially.

In addition, bank conflicts pose a significant bottleneck when LUT-based methods are deployed on existing hardware platforms. For instance, LUT-GEMM [28] stores the LUTs in the shared memory of GPU and utilizes multiple threads for operations. During the LUT construction phase, bank conflicts are avoided as each thread is designed to access different banks in parallel. In contrast, during the LUT read phase, the randomness of the weight pattern often causes frequent bank conflicts, leading to performance degradation. Fig. 2 depicts the example of multiple threads accessing memory banks. In the worst-case scenario, all required accesses within a cycle are serialized, creating substantial performance overhead. Therefore, for LUT-based operations to achieve optimal performance, it is essential to avoid bank conflicts. This requires a novel LUT architecture that allows multiple parallel operators to simultaneously read different values without conflicts.

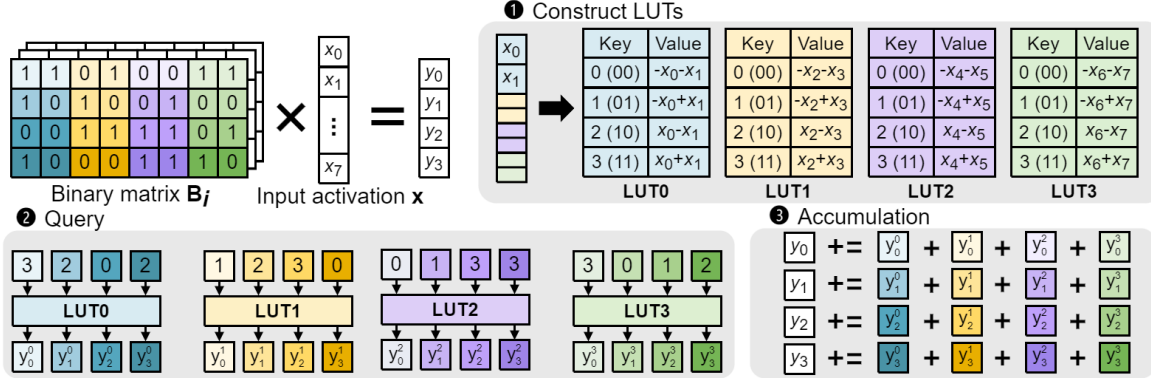


Fig. 3. Illustration of look-up table based FP-INT GEMM.

TABLE II
EXAMPLE OF LOOK-UP TABLE WHEN $\mu = 3$

Binary Patterns	Key	Value
$\{-1, -1, -1\}$	0 (b'000)	$-x_1 - x_2 - x_3$
$\{-1, -1, +1\}$	1 (b'001)	$-x_1 - x_2 + x_3$
$\{-1, +1, -1\}$	2 (b'010)	$-x_1 + x_2 - x_3$
$\{-1, +1, +1\}$	3 (b'011)	$-x_1 + x_2 + x_3$
$\{+1, -1, -1\}$	4 (b'100)	$+x_1 - x_2 - x_3$
$\{+1, -1, +1\}$	5 (b'101)	$+x_1 - x_2 + x_3$
$\{+1, +1, -1\}$	6 (b'110)	$+x_1 + x_2 - x_3$
$\{+1, +1, +1\}$	7 (b'111)	$+x_1 + x_2 + x_3$

III. LOOK-UP TABLE-BASED FP-INT GEMM ACCELERATOR

In this section, we introduce FIGLUT, a novel computational methodology and accelerator design aimed at improving the efficiency of FP-INT GEMM operations by replacing computation with data retrieval from an LUT. To address the issue of LUT bank conflicts that hinder parallel processing in existing LUT-based operations, we propose a specialized LUT structure. This structure is designed to eliminate bank conflicts and enable efficient parallel access to LUT data. We also analyze the impact of increased fan-out on power consumption in shared LUTs and propose an optimal processing element (PE) structure to maximize energy efficiency. As a result, the FIGLUT design offers enhanced energy efficiency relative to conventional computing units.

A. Look-up Table based FP-INT GEMM

Look-up tables are extensively employed to improve computational efficiency by substituting direct calculations with table look-ups, particularly when the results are confined to a predetermined set. This technique can be extended to deep learning operations involving a binary weight matrix $\mathbf{B} \in \{-1, +1\}^{M \times N}$, where output activations are calculated through simple addition and subtraction among the input activation vector $\mathbf{x} \in \mathbb{R}^{N \times 1}$. Consequently, the output activations are confined to combinations of input activations, enabling the

calculation and storage of all potential output values within the LUT. This approach substitutes actual computation with the retrieval of data from the LUT.

To construct an LUT, we introduce a hyperparameter μ , which determines the number of binary weights used to form a key for retrieving a value from the LUT. Note that as μ increases, the number of computations that can be replaced by a single LUT read also increases. However, since the memory size required to store the LUT grows exponentially, finding the optimal value of μ is crucial. Consider a binary matrix $\mathbf{B} \in \{-1, +1\}^{4 \times 6}$ and an input activation vector $\mathbf{x} \in \mathbb{R}^{6 \times 1}$:

$$\mathbf{B} = \begin{bmatrix} +1 & -1 & -1 & -1 & -1 & +1 \\ +1 & -1 & -1 & +1 & +1 & -1 \\ -1 & +1 & -1 & -1 & -1 & +1 \\ +1 & -1 & -1 & -1 & -1 & +1 \end{bmatrix}, \quad (4)$$

$$\mathbf{x} = [x_1 \ x_2 \ x_3 \ x_4 \ x_5 \ x_6]^\top. \quad (5)$$

For instance, in the computation of $\mathbf{B}\mathbf{x}$ with $\mu = 3$, we observe that operations such as $(x_1 - x_2 - x_3)$ and $(-x_4 - x_5 + x_6)$ are repeated multiple times. These redundant computations increase as the size of the matrices grows with the increasing model size. As shown in Table II, by pre-computing all possible combinations of inputs and storing them in the LUT, repeated computations can be avoided, allowing a single LUT read to replace multiple addition operations. If μ is increased to 6, there are repeated computations such as $(x_1 - x_2 - x_3 - x_4 - x_5 + x_6)$, which can also be efficiently managed using the LUT. Although increasing μ enhances computational benefits, the memory space required to store the LUT increases exponentially.

Fig. 3 provides an overview of the LUT-based FP-INT GEMM process for computing $\mathbf{B}\mathbf{x}$ with $\mu = 2$. As previously mentioned, to reduce redundant computations and efficiently perform GEMM operations, LUTs are constructed using μ elements from the input activation $\mathbf{x}$, customized for each input. Each LUT contains 2^μ values, with μ binary weights used as keys to retrieve each value. During the query phase,

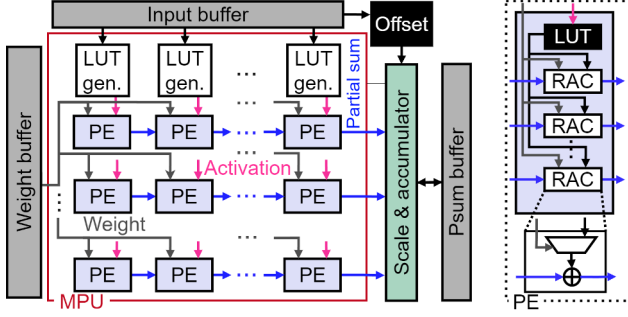


Fig. 4. Overall MPU architecture of FIGLUT

k RACs simultaneously access the shared LUT to retrieve their respective partial sum values. In this example, $k = 4$, indicates that four RACs can access a single LUT concurrently. The specialized LUT structure for LUT-based GEMM, which enables concurrent access by k RACs, will be detailed in the subsequent section. Finally, the values retrieved from each LUT are stored in the accumulator. Once this process is completed for all binary matrices $\mathbf{B}_i$, the final output activation values are obtained. Note that while the previous examples assume the input activation to be a vector for simplicity (corresponding to a GEMV operation), this approach is basically applied to GEMM operations by generating unique LUTs for each batch, allowing for efficient matrix computations.

B. Overall FIGLUT Architecture

Fig. 4 depicts the overall architecture of the proposed LUT-based FP-INT GEMM accelerator, FIGLUT. In this section, we provide a detailed description of the matrix processing unit (MPU), which represents our primary contribution. A comprehensive overview of the entire system architecture will be discussed in Section III-F. FIGLUT leverages a 2D systolic array structure similar to that of the Google TPU [18]. Inputs are fetched sequentially from the input memory through systolic data setup. These input activations are fed into the LUT generators to construct the LUTs. Each LUT generator takes μ input activations and generates all possible output partial sums on-the-fly. These partial sums are then passed to the LUTs within the PEs, where they are used as LUT values. To enable effective data reuse, LUT values from the PEs in each row are forwarded to the LUTs in the subsequent rows. Each PE houses k RACs, and due to the unique structure of LUT, designed for LUT-specific operations, the k RACs can read from the LUT concurrently. We perform an analysis to determine the optimal value of k , taking into account the fan-out overhead introduced when multiple RACs access the LUT at the same time. This analysis is elaborated in a later section.

FIGLUT employs a weight-stationary dataflow to optimize GEMM operations. Within each PE, the RAC includes a register that holds a μ -bit weight pattern, serving as a key to access data from the LUT. The partial sums generated by each PE are incrementally gathered and subsequently sent to the PE in the next column. This iterative accumulation process takes

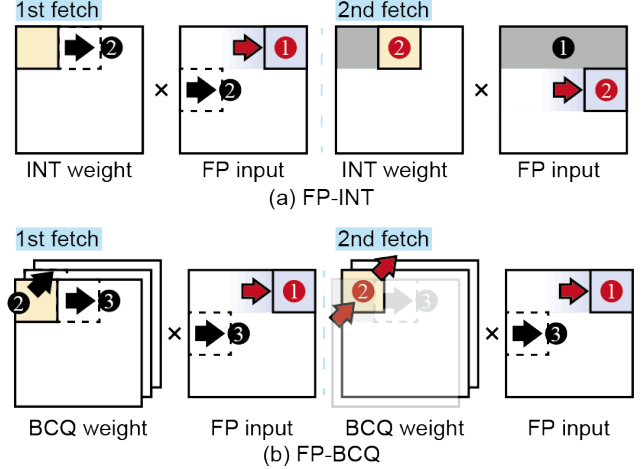


Fig. 5. Illustration of the input and weight tile fetching sequence in (a) a systolic array accelerator for INT weight and (b) FIGLUT for BCQ weight. The arrows indicate the order in which the tiles are processed.

place along each column of PEs until the ultimate partial sum is derived at the final PE. The eventual partial sum is multiplied by a scaling factor α_i and stored in the accumulator buffer. This cycle is performed for each bit plane, and once all bit planes are processed, the accumulated sums are summed with the offset value and routed to the output buffer.

Fig. 5 shows the fetching sequence of input and weight tiles. For systolic array accelerators with INT weights, such as FIGNA [16], each weight has a single channel with multi-bit values. In contrast, FP-BCQ accelerators like FIGLUT and iFPU [22] include q binary weight bit planes. In a weight-stationary dataflow, a weight tile is loaded once and reused, while inputs are loaded sequentially (1) for both FP-INT and FP-BCQ accelerators. However, the process diverges for FP-BCQ accelerators. Instead of loading the next tile within the same bit plane, the weight tile for the next bit plane is loaded and processed (2 in Fig. 5 (b)). After processing all binary weight bit planes, the operation proceeds to the next tile, similar to the FP-INT accelerator.

C. Conflict-free LUT Structure

To efficiently perform LUT-based FP-INT GEMM operations, a specialized LUT structure is essential, as traditional memory structures are insufficient for the following requirements. First, minimizing the power consumption associated with reading data from the LUT is crucial. Since FIGLUT replaces computation with data retrieval, reducing the cost of data reads compared to the cost of computations enhances overall efficiency. Second, the structure needs to support a large volume of simultaneous read and write operations. Given the dynamic nature of activation values, the LUT should be reconfigurable on-the-fly, allowing concurrent writing of large amounts of data to update the LUT according to input activations. From a data retrieval perspective, multiple RACs operate in parallel, each using different weight patterns as keys

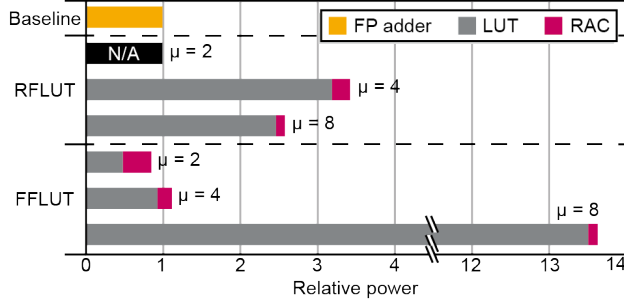


Fig. 6. Relative Power Consumption of RFLUT and FFLUT Compared to FP Adder Baseline Across Different μ Values

to access data in the LUT. Therefore, the architecture should facilitate parallel data access while avoiding bank conflicts.

To ensure that retrieving precomputed results from an LUT consumes less power than using FP adders, we measure the power consumption of an FP adder as a baseline. We then compare this with the power consumption of LUT-based operations. Fig. 6 demonstrates the power comparison between FP adders and various LUT implementations across different μ values, while maintaining equivalent throughput. To address the targeted hardware constraints, we implemented two types of LUTs: (1) a traditional Register File based LUT (RFLUT) and (2) a proposed Flip-Flop based LUT (FFLUT).

The RFLUT, constructed using register files commonly employed in CPU register arrays, uses a straightforward approach where the weight pattern serves as the read register number. For the comparison, RFLUT blocks are generated using a 28nm memory compiler, allowing us to customize the design to target low-power LUT-based GEMM operations. The generated register file is then integrated into the hardware design as an IP macro block. All power analyses of LUT structures, including FFLUT, are tested based on the physical layout obtained after the P&R process in ICC2. As the RFLUT with $\mu = 2$ is too small to be generated by the compiler in this environment, it is not considered for measurement. Generally, RFLUTs exhibit higher read power consumption compared to FP addition operations. While an RFLUT with $\mu = 4$ requires less power per read than one with $\mu = 8$, it needs twice the number of reads, leading to higher overall power consumption. As a result, RFLUT is not suitable for the LUT-based method, and a novel method is necessary to construct LUTs.

To overcome the limitations of traditional RFLUTs, we propose a Flip-Flop-based Look-Up Table (FFLUT) structure, illustrated in Fig. 7. The FFLUT consists of a bundle of flip-flops accessed through multiplexers. Each enabled flip-flop outputs consistent values once updated during the LUT generation phase, maintaining this output until it is reset. This design allows the processing unit to retrieve values by multiplexing the corresponding weight keys to access the required flip-flops. Unlike RFLUTs, which are constrained by their limited read/write ports, the FFLUT enables simultaneous access by multiple units without encountering bank conflicts.

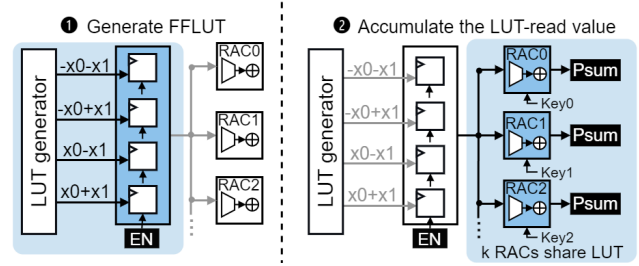


Fig. 7. Architecture of the Flip-Flop based Look-Up Table (FFLUT)

This is achieved by equipping the FFLUT with dedicated multiplexers that allow different units to access distinct keys concurrently. As a result, multiple units can share a single LUT efficiently, significantly reducing the overhead associated with LUT-based operations and improving energy efficiency.

To further optimize the data retrieval process in FFLUTs, we introduce a specialized operator called the RAC unit, designed as a replacement for the conventional MAC unit. Each PE in FIGLUT is equipped with a single LUT and multiple RAC units, as shown in Fig. 4. Unlike the MAC unit, which performs a multiplication followed by an accumulation into a partial sum, the RAC unit retrieves values directly from the LUT using a weight pattern as a key and accumulates them into the partial sum. Hence, the RAC unit can execute LUT-based operations more effectively, enhancing the overall efficiency of the FIGLUT architecture.

To achieve energy-efficient GEMM operations, our approach incorporates an optimal architecture search that allows multiple RAC units to share a single LUT. This optimization process involves two key steps. First, we determine the optimal μ value for the LUT, balancing the gains of the computation against the power consumption of the LUT. Second, based on the chosen μ value, we identify the optimal number of RACs per LUT by evaluating the impact of fan-out on power consumption. We introduce a variable k to represent the LUT fan-out within a single PE, denoting the number of RACs sharing a single LUT. Fig. 8 shows the power consumption of FIGLUT with μ values of 2 and 4, varying k . The baseline for relative power measurements is set using the results of the same operations performed with FP adders. Due to the significantly large size and high power consumption of LUTs for $\mu = 8$, as shown in Fig. 6, this configuration is excluded from consideration. As k increases, the total number of LUTs decreases by sharing an LUT with multiple RACs, leading to a gradual reduction in LUT power consumption. Note that the total number of RACs required for computation remains constant regardless of k , as it depends solely on the μ value. This is because the overall number of read-accumulate operations is unaffected by the sharing configuration. However, smaller μ values require more RACs, resulting in higher RAC power consumption for $\mu = 2$ compared to $\mu = 4$. When LUTs are not shared and are exclusively used by a single RAC ($k = 1$), the larger LUT size for $\mu = 4$ leads to

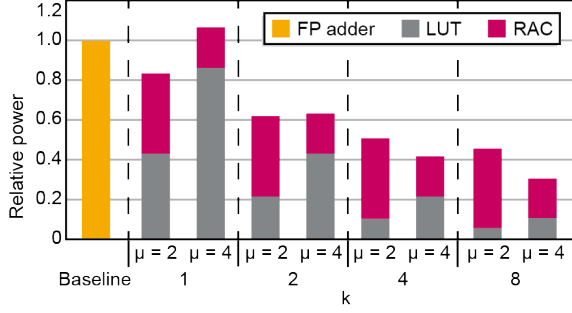


Fig. 8. Relative power comparison of baseline, $\mu = 2$ and $\mu = 4$ for various the number of RACs per LUT configuration.

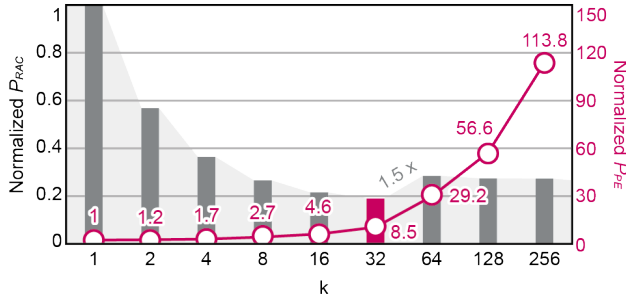


Fig. 9. P_{RAC} and P_{PE} analysis for various RAC numbers. Normalized by $k = 1$ results.

higher power consumption, resulting in higher total power consumption compared to $\mu = 2$. However, as k increases and LUTs are shared among multiple RACs, the number of LUTs decreases, reducing overall power consumption. Based on these findings and considering sufficiently large k values in our architecture, we use the FIGLUT architecture with $\mu = 4$ in this paper.

Managing fan-out is a critical challenge in designing an energy-efficient accelerator capable of sharing a single LUT among multiple RACs. In digital circuits, fan-out occurs when a signal is distributed to multiple circuit elements, leading to increased power use and signal delay. In the context of FIGLUT, as multiple RACs access the LUT simultaneously, the fan-out effect requires additional power to deliver signals to each RAC efficiently. While sharing a single LUT among multiple RACs reduces the total number of LUTs, it also increases the power consumption of each LUT due to the heightened fan-out. This trade-off necessitates careful management to achieve an optimal balance between the benefits of reducing the total number of LUTs and the drawbacks of increased power demand from fan-out. Effectively addressing this issue is crucial for minimizing overall power use while maintaining energy efficiency in LUT-based GEMM operations.

Fig. 9 shows the power consumption of the PE (P_{PE}) and each RAC (P_{RAC}) as a function of LUT fan-out. To measure the impact of fan-out, all experimental results are obtained using the physical layout of the PEs (including LUTs and

TABLE III
COMPARISON OF RELATIVE POWER CONSUMPTION OF LUT AND OTHER COMPONENTS

	LUT	MUX	Decoder	MUX+Decoder
FFLUT	1.000	0.003	0.000	0.003
hFFLUT	0.494	0.002	0.003	0.005

RACs) after the P&R process. The power consumption per RAC unit P_{RAC} is calculated by dividing P_{PE} by k . As k increases, the number of RACs within the PE increases, resulting in higher overall PE power. Initially, increasing k reduces P_{RAC} because the power consumption of the shared LUT is distributed across the k RACs. However, as k continues to grow, the fan-out overhead becomes significant, increasing P_{RAC} . Based on the observation, we determine the optimal value of k to be 32 for configuring PEs in our architecture. Thus, each PE in our architecture includes one shared LUT and 32 RACs.

D. Optimizing LUT using Vertical Symmetry

In LUT-based FP-INT GEMM, the primary overhead compared to conventional GEMM operations is the LUT itself. Reducing LUT power consumption is therefore critical for enhancing the efficiency of FIGLUT. The FFLUT is composed of flip-flops, with power consumption scaling linearly with its size. To address this, we propose a half-FFLUT (hFFLUT) technique, which reduces the LUT size by half.

As shown in Table II, LUT values exhibit vertical symmetry. In other words, since the LUT is composed of the calculation results of μ FP values through combinations of addition and subtraction, for every combination, there exists a corresponding value that can be directly obtained by simply flipping the sign. By exploiting this characteristic, we store only the upper half of the table and use the key to decode the final LUT value. Fig. 10 presents a block diagram illustrating the hFFLUT and the proposed decoding technique when $\mu = 3$. The most significant bit (MSB) of the key is used as a select signal to choose the $(\mu - 1)$ -bit key for the hFFLUT. Then, the final value read from the hFFLUT is then sign-flipped based on the MSB to obtain the complete value.

Although the hFFLUT effectively reduces the size of the LUT, it introduces additional decoding overhead. Table III evaluates the hardware complexity of the proposed decoding circuit for hFFLUT compared to the FFLUT multiplexer used to fetch data from the LUT. The decoding circuit for hFFLUT, including the multiplexer, demonstrates higher power consumption than the multiplexer used in the FFLUT. However, since LUT power is dominant in the overall power consumption, the additional overhead of the decoding circuit is relatively trivial in comparison. Consequently, the proposed hFFLUT effectively halves the power consumed by the LUT in the entire system, thereby significantly enhancing the energy efficiency of FIGLUT.

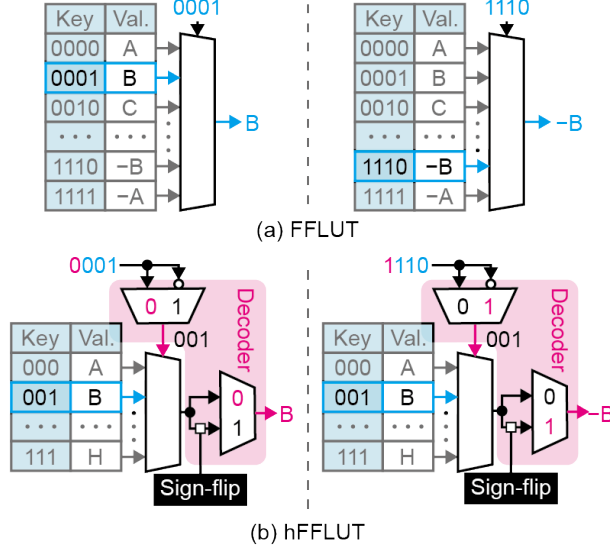


Fig. 10. Block diagram of the (a) FFLUT and (b) hFFLUT with proposed decoder.

E. Efficient LUT Generator

An efficient table generator that dynamically adapts to changing input activations is essential to calculate LUT elements. We implemented a two-step tree-like hardware module that changes the sign of inputs and sums them up in parallel, generating multiple LUT values simultaneously. By leveraging the hFFLUT, which halves the number of LUT elements, the generator is designed to pre-compute results for only half the patterns. As shown in Fig. 11, the lower bit patterns repeat for specific upper bit patterns (yellow shaded), allowing us to minimize the addition operations for the upper bit patterns. Duplicated lower bit parts (green shaded) are calculated once and connected in parallel to the next adders. By combining the results of upper 2-bit patterns with the lower 2-bit patterns, we can generate all the necessary hFFLUT patterns. This approach reduces the number of adders and the total addition operations by 42% compared to a straightforward implementation of a LUT generator. Assuming all LUT elements are used, for $\mu = 4$, the LUT generator requires 14 additions to compute the complete set of results. In contrast, straightforward hardware without an LUT requires $\mu - 1 = 3$ additions per result. Thus, for $k > 4$, the proposed LUT generator performs fewer additions to compute the same number of results compared to straightforward hardware with k RACs. As k increases, the addition saving of the proposed method becomes even more prominent, enabling a more efficient LUT generation.

F. System overview

Fig. 12 provides an overview of the entire system, including FIGLUT. As a hardware accelerator module, FIGLUT is designed to be flexible and is not limited to specific system configurations, enabling seamless integration into a wide range of system architectures beyond the proposed setup.

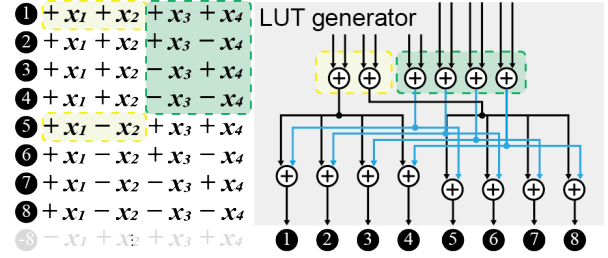


Fig. 11. Required generating pattern and LUT generator module for $\mu = 4$.

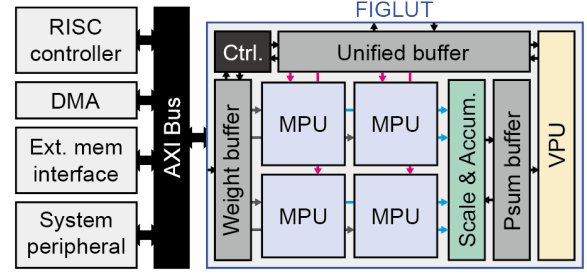


Fig. 12. System architecture of FIGLUT.

The system adopts a shared memory structure between the host and FIGLUT, rather than using separate memory spaces. Data transfer between the host and FIGLUT is facilitated through the AXI bus. This architecture eliminates the need to explicitly transfer final computation results from FIGLUT to the host; instead, the host can directly access the shared memory to retrieve the results efficiently. It minimizes the overhead associated with data movement between FIGLUT and the host, thereby maximizing overall system efficiency.

FIGLUT comprises on-chip buffers for storing computation data, along with MPUs and a vector processing unit (VPU) to efficiently manage non-GEMM operations. To maximize efficiency, FIGLUT adopts a tile-based GEMM operation approach, where the data required for each tile computation is stored in the on-chip buffer using double buffering techniques. The MPU executes LUT-based GEMM operations, storing the intermediate computation results as partial sums in the Psum buffer for subsequent usage. The final GEMM results are then processed by the VPU, and the output data is stored in a unified buffer. This approach minimizes off-chip memory access and maximizes system efficiency.

IV. EVALUATION

A. Accuracy Evaluation

To evaluate the numerical accuracy of the proposed FIGLUT, we conduct an inference accuracy assessment using both an NVIDIA GPU and the FIGLUT engine. To further optimize our engine, we utilize the pre-alignment technique [16], [22] used in FP-INT GEMM to FIGLUT, resulting in two variations: FIGLUT-F, which bypasses the pre-alignment technique, and FIGLUT-I, which incorporates it. We evaluate the transformer-based OPT model family [39] on the WikiText-2

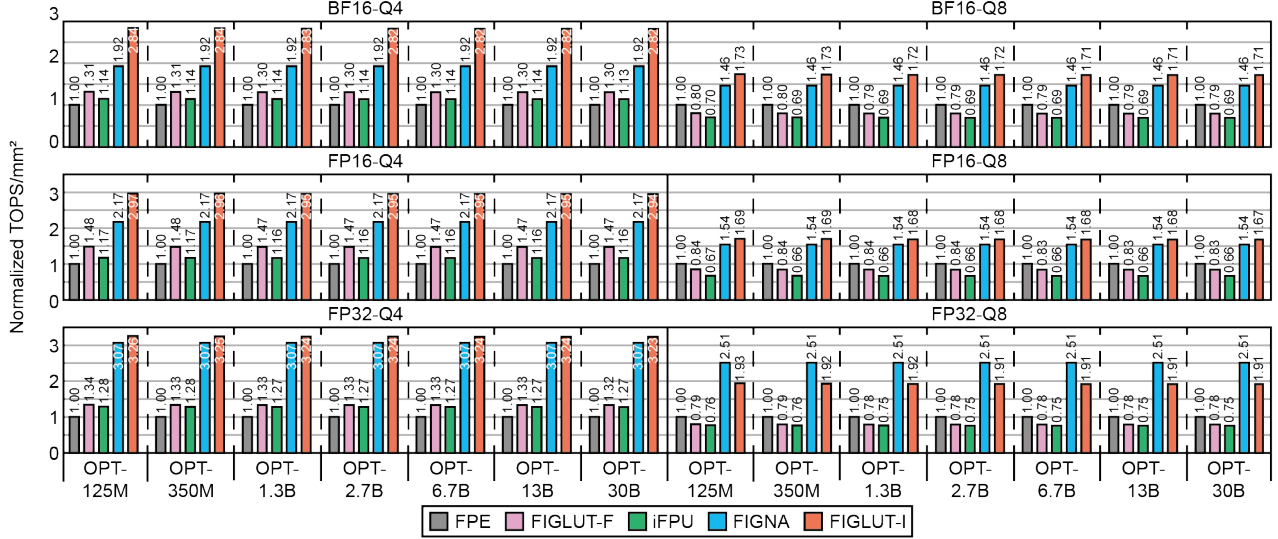


Fig. 13. TOPS/mm² of hardware engines for Q4 and Q8 language models, normalized by FPE results.

TABLE IV
PERPLEXITY RESULTS OF THE OPT FAMILY MODELS USING DIFFERENT GEMM ENGINES.

OPT	350M	1.3B	2.7B	6.7B	13B	30B
GPU	55.24	67.95	35.46	24.13	20.93	19.17
FIGLUT-F	55.24	67.95	35.46	24.13	20.93	19.17
FIGLUT-I	55.24	67.95	35.46	24.13	20.89	19.17

dataset [27] for a language modeling task, measuring perplexity as the metric. All models use FP16 activations and 4-bit weights, with the weights quantized using the simple uniform quantization method, round-to-nearest (RTN). Note that we use FP32 for accumulation to preserve the accuracy of the accumulated results [6], [13]. Table IV shows the perplexity results to compare the numerical accuracy of each GEMM engine against the NVIDIA GPU. Although the order of FP operations may change, potentially leading to differences in values, the results for FIGLUT-F show no significant accuracy loss compared to the NVIDIA GPU, thanks to the higher precision (FP32) accumulation. FIGLUT-I also demonstrated comparable numerical accuracy with pre-alignment method, consistent with findings reported in previous studies [16], [22].

B. Hardware Evaluation

a) *Configuration Setup*: We evaluate five types of hardware engines: FIGLUT-F, FIGLUT-I, FPE, iFPU [22], and FIGNA [16]. The FPE serves as the baseline hardware engine, featuring PEs that dequantize INT weights to FP format with the same precision as the activation inputs, followed by FP multiplication and FP accumulation. For both iFPU and FIGNA, the engines carry out pre-alignment procedures to transform FP input activations into integer mantissas with a common exponent prior to performing MAC operations. For

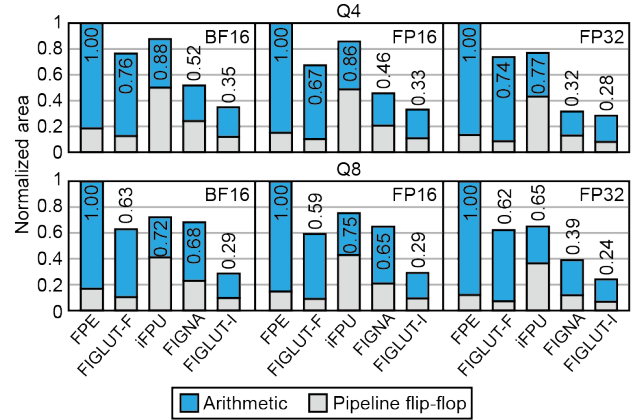


Fig. 14. Area breakdown of MPUs for six variants of input formats normalized by results of FPE.

comparative analysis with the engines, the proposed FIGLUT hardware is implemented in two versions with different processing data formats: FIGLUT-F, which uses FP inputs directly into the LUT element, followed by FP RAC, and FIGLUT-I, which performs integer operations after pre-alignment. All hardware engines are designed to handle inputs with the same bit precision and word size and produce outputs in the same manner. Notably, iFPU and FIGLUT support the BCQ format, whereas FPE and FIGNA utilize the INT format. To conveniently reference the different weight formats each hardware uses, we denote them by their bit-widths, using a notation such as Q4 for both 4-bit BCQ and INT weight.

For fair comparison, all engines are designed with MPUs and VPUs that provide identical throughput. Specifically, for Q4 weight inputs, FPE and FIGNA have 64×64 PE arrays,

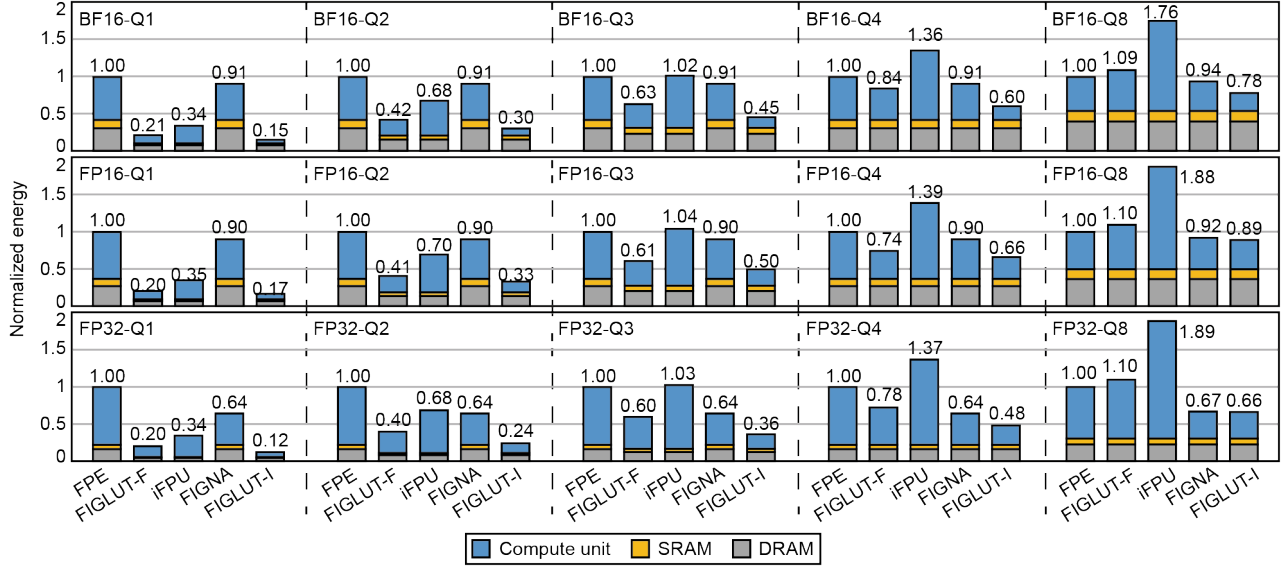


Fig. 15. Normalized energy breakdown of FP-INT GEMM hardware engines across different bit precisions.

while iFPU processing 1-bit weight employs a $64 \times 64 \times 4$ array. FIGLUT uses an $2 \times 16 \times 4$ array, which, considering $\mu = 4$ and $k = 32$, results in the same number of computational units as iFPU. For weights with a bit precision smaller than Q4, the same hardware configurations are used. For Q8 weights, results are obtained using extended versions of the FPE and FIGNA hardware, where the input weight bit precision was expanded to 8-bit. To leverage the bit-serial characteristics of iFPU and FIGLUT, both hardware engines are configured to match the throughput of Q4, affecting the operation cycles required for matrix multiplication for other weight precision.

The hardware engines are synthesized with Synopsys Design Compiler for 100MHz operation with 28nm CMOS technology. Basic FP and integer modules, including the integer-to-FP converter in FPE, use Synopsys DesignWare components. All input, weight, and output buffers are composed with SRAM in 28nm CMOS technology, with off-chip DRAM results derived from the CACTI simulator [5]. Data from the external DRAM is transferred directly to the FIGLUT accelerator SRAM buffer under the management of the host controller. The data is subsequently utilized by the MPU and VPU for computations. Energy metrics, including power consumption and latency associated with memory data transfers, are incorporated into all evaluation results, with the exception of area measurements, as the DRAM area is unspecified.

b) Area Evaluation: Fig. 14 illustrates the normalized area breakdown of the MPUs across varying engine types, considering different input precision scenarios. The breakdown is categorized into two primary components: arithmetic logic, responsible for executing computations, and flip-flops to temporarily store values. The result shows that the arithmetic part occupies a larger area in FPE and FIGLUT-F compared to other integer-operating engines, with FIGLUT-F having

a smaller area than FPE because it performs FP addition instead of FP multiplication. In the Q8 results, the arithmetic area of FIGNA increases more significantly compared to FPE because the computational units in FIGNA scale with the bit precision of the weights, whereas FPE only encounters an increase in the overhead of the dequantization module without a change in the actual input bit-width of FP operators. When comparing FIGLUT-I and FIGNA, despite the addition of an LUT generator, the use of LUT-based operations reduces the number of computational units, resulting in a similar arithmetic area as FIGNA. Additionally, the introduction of LUT-based operations reduces the overall flip-flop area compared to other hardware architectures. With a smaller $2 \times 16 \times 4$ MPU, compared to the conventional configurations of 64×64 and $64 \times 64 \times 4$, FIGLUT effectively reduces the number of pipelining stages required for systolic array operations. Furthermore, while traditional hardware designs may demand up to 63-stage input buffers for weight-stationary operations, FIGLUT requires a maximum of only 15-stage input buffers, enhancing both efficiency and resource utilization.

Fig. 13 presents the area efficiency targeting various LLMs ranging from OPT-125M to OPT-30B. Excluding hardware designs with bit-serial architecture, the overall trends inversely follow the area breakdown shown in Fig. 14 across all network and input bit-widths. For instance, as previously noted, the PE size of the FIGNA is relatively larger in Q8 compared to Q4, resulting in reduced normalized energy efficiency. However, the bit-serial engines exhibit different results because TOPS/mm^2 reflects the time required for computations. With the same hardware configuration, hardware designs with bit-serial architecture consume approximately twice the cycles with increased weight bit-width, leading to more significant performance degradation in Q8. The difference between

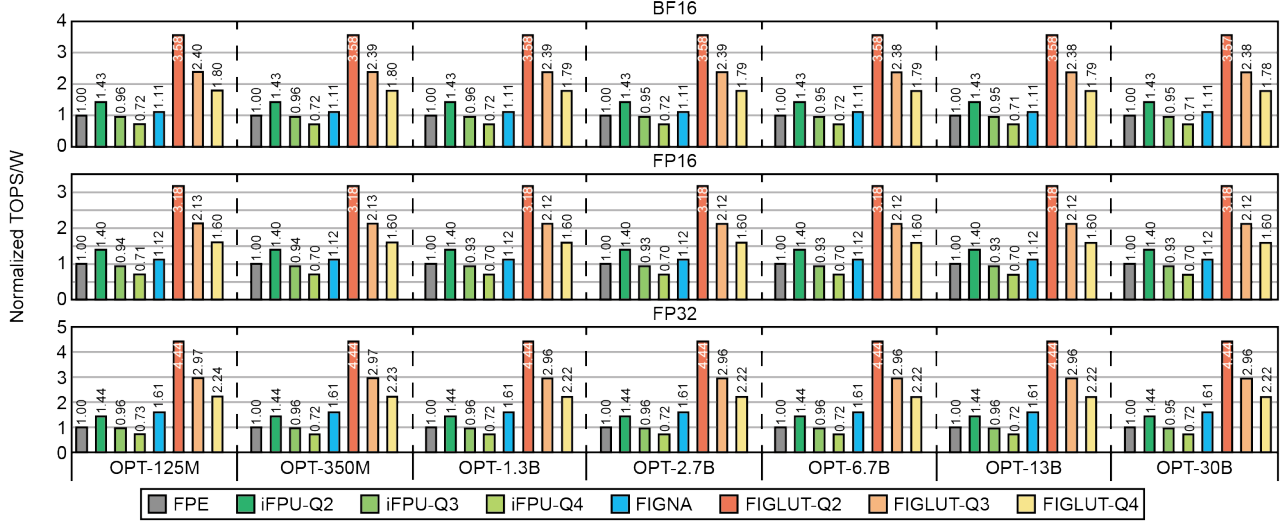


Fig. 16. TOPS/W of hardware engines for under 4-bit OPT language models, normalized by FPE results.

FIGNA and FIGLUT-I decreases for FP32 due to the increased mantissa bit-width, contributing to performance reversal in FP32-Q8. Nonetheless, the proposed engines achieve up to $1.5\times$ higher area efficiency than state-of-the-art performance in the current trend of sub-4-bit weight-only quantization. Note that non-GEMM operations cause slight variations by model size, but their impact is minimal as GEMM operations dominate the workload.

c) *Energy Evaluation:* Fig. 15 shows the energy breakdown for each input data type pair, normalized to the energy consumption of the FPE. To assess and compare the energy efficiency across various bit precisions—including Q4, Q8, and sub-Q4 precisions (Q1, Q2, Q3)—we conduct simulations using the OPT-6.7B model. As previously mentioned, all systems are configured to have the same maximum performance to ensure a fair comparison. In Q8 operations, while bit-serial hardware continues to utilize identical architecture irrespective of bit precision, fixed precision architectures like FPE and FIGNA are modified to support 8-bit weights. For sub-4-bit weight precision, the number of bit-serial operations decreases proportionally as the bit-width is reduced, whereas fixed bit-width hardware needs to process sub-4-bit data in padded 4-bit format for computation. This allows bit-serial hardware such as iFPU and FIGLUT to complete computations more quickly, effectively reducing energy consumption as weight precision decreases. The energy consumption of the compute unit, including the MPU and the VPU, for Q4 precision follows a trend similar to the area breakdown depicted in Fig. 14. However, iFPUs, which employ a greater number of flip-flops than FPEs, suffer from higher power consumption despite being more area-efficient, resulting in a distinct energy profile. In the case of FIGLUT-I, the size of the PE increases from BF16 to FP32 because the computational units scale proportionally with the mantissa of the input activation.

Fig. 16 illustrates the power efficiency of the hardware en-

gines when performing sub-4-bit weight operations on LLMs of various sizes. To highlight the performance variations in bit-serial operation engines, we present power efficiency metrics for Q2, Q3, and Q4 operations. Given that FIGLUT-I shows better power efficiency with integer operations compared to FIGLUT-F, we focus solely on FIGLUT-I for this evaluation. As expected, reducing the weight bit-width decreases the number of cycles required for computations in bit-serial hardware, thereby improving energy efficiency. FIGLUT consistently achieves the highest TOPS/W across all weight bit-widths, with FIGLUT-Q2 demonstrating particularly superior performance gain compared to other configurations. This result explains that in memory-bound computing environments, i.e., LLMs, minimizing the weight bit-width and leveraging bit-serial operations can significantly enhance computational speed, resulting in substantially higher power efficiency compared to traditional hardware.

Fig. 17 shows the TOPS/W performance and perplexity results of FIGLUT across various mixed-precision configurations for the OPT-6.7B LLM. We compare these results against the baseline FIGNA, evaluated using the OPTQ quantization method [10] at 2, 3, and 4-bit precisions. In contrast, FIGLUT, including mixed-precision performance, is assessed using ShiftAddLLM [36]. Note that recent advancements in weight-only quantization techniques [7], [10], [14], [19], [24], [25], [36], [38] have enabled sub-4-bit models to achieve performance levels comparable to the unquantized FP16 baseline, as indicated by the dotted lines in the figure. Under the same 4-bit quantization, FIGLUT is $1.2\times$ more energy efficient with slightly better perplexity than FIGNA. For Q3, FIGLUT achieves $1.6\times$ the energy efficiency of FIGNA and records a lower perplexity. Furthermore, at a similar energy efficiency level, FIGLUT-Q2.4 demonstrates $1.98\times$ the computational efficiency of FIGNA-Q3 while compressing the model size

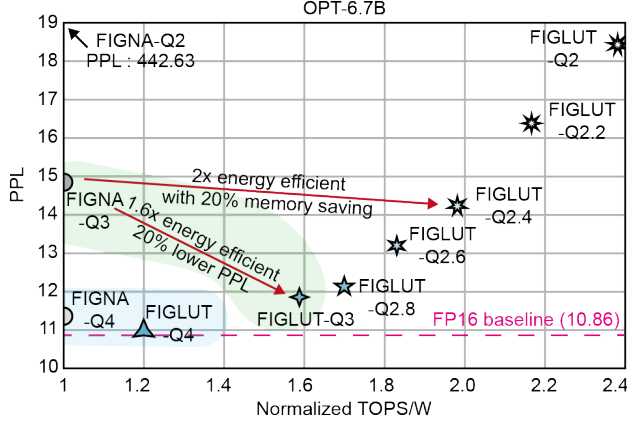


Fig. 17. TOPS/W and perplexity score for various mixed-precision OPT-6.7B inference on FIGLUT.

TABLE V
COMPARISON OF HARDWARE ACCELERATOR DESIGNS.

Hardware	Format (Act.-Weight)	Throughput (TOPS)	Power (W)	Energy Effi. (TOPS/W)
A100	FP16-FP16	40.27*	192	0.21*
A100	FP16-Q4**	1.85	208	0.01
H100	FP16-FP16	62.08*	279	0.22*
iFPU	FP16-Q4	0.14	0.67	0.21
FIGNA	FP16-Q4	0.14	0.41	0.33
FIGLUT	FP16-Q4	0.14	0.29	0.47

* The unit for GPUs is TFLOPS.

** LUT-GEMM kernel [28] is used.

by 20%. For 2-bit weight precision, FIGLUT supports a non-uniform BCQ format, delivering much more stable perplexity performance compared to FIGNA, and improving energy efficiency by up to $2.4\times$.

Table V shows the energy efficiency of various hardware accelerators. Since GEMM operations dominate LLM inference workloads [28], we focus on comparing GEMM performance. For a fair comparison, we include state-of-the-art commercial accelerators, the A100 and H100 GPUs, additionally. As GPU architecture details are not publicly disclosed, we rely on empirical measurements for evaluation. GPU throughput is based on the computational workload and measured latency, while power consumption is obtained using the `nvidia-smi` query tool [2], [28], [30]. The target LLM model is OPT-6.7B, with a batch size of 32 for all evaluations to reflect realistic LLM usage. To ensure fairness, we assume a 4-bit quantized weight format across all tests. Note that, due to the absence of FP-INT units in GPUs, they rely on FP-FP units for computation with dequantization. Although LUT-GEMM [28] proposes an efficient FP-INT kernel, it only supports a batch size of 1. To standardize comparisons, we align all FP-Q4 accelerators to target the same maximum performance (TOPS).

The H100 outperforms the A100 in terms of energy efficiency, primarily due to its advanced fabrication technology

TABLE VI
PERPLEXITY COMPARISON FOR WEIGHT-ONLY QUANTIZATION.

OPT	350M	1.3B	2.7B	6.7B	13B	30B
FP16	22.00	14.62	12.47	10.86	10.13	9.56
BCQ4 [36]	22.59	15.11	12.73	11.08	10.33	9.70
BCQ3 [36]	28.72	19.69	15.28	11.80	10.70	9.89

and increased memory bandwidth, which significantly boosts performance in memory-bound workloads like LLMs. Note that the reported TFLOPS for GPUs are significantly lower than their theoretical peaks, primarily due to the small batch size. In LLM inference infrastructure, batch sizes larger than 32 are rarely observed at a generation phase due to constraints in KV cache size and the optimization of batching systems for user requests [23], [37]. LUT-GEMM suffers from an even more pronounced performance drop as it operates with a batch size of 1 and relies on CUDA cores rather than Tensor Cores. The iFPU, while designed to maximize efficiency with FP-INT units, demonstrates limited performance compared to the H100 due to the inherent constraints of bit-serial processing. FIGNA addresses these limitations, resulting in higher energy efficiency. Finally, FIGLUT further improves energy efficiency by overcoming the computational challenges of bit-serial processing through its LUT-based GEMM approach. Given that FIGLUT uses a 28nm process technology while GPUs like the A100 and H100 leverage more advanced fabrication technology (7nm for A100, 4nm for H100), the efficiency of FIGLUT would be even more prominent if evaluated under comparable fabrication technologies.

V. CONCLUSION

Summary. This paper presents FIGLUT, an FP-INT accelerator that enhances the efficiency of bit-serial accelerators for weight-only quantized models using LUT-based operations. We introduce a flip-flop based LUT (FFLUT) and replace the traditional MAC unit with a read-accumulate (RAC) unit for optimized performance. Our analysis of LUT size and RAC configuration leads to an energy-efficient design that halves the LUT size through symmetry. Experiments show FIGLUT outperforms state-of-the-art FP-INT accelerators in TOPS/W for sub-4-bit precision weights, significantly improving efficiency while maintaining flexibility in real-world systems.

Limitations. As shown in Fig. 15, our design shows diminishing performance gains as weight bit precision increases, a limitation of the bit-serial architecture. To achieve high performance with larger bit precisions, increasing μ would be necessary to fully leverage the computational benefits of LUT-based GEMM. However, this results in much larger LUTs, increasing overhead in the LUT and its generator. Nevertheless, Table VI indicates that state-of-the-art weight-only quantization techniques have enabled sub-4-bit quantization while maintaining accuracy comparable to the FP16 baseline. We expect that continued advancements in weight-only quantization methods will further enhance the effectiveness of the proposed FIGLUT as bit precision is reduced.

VI. ACKNOWLEDGMENT

This research was supported in part by the NAVER-Intel Co-Lab, conducted by POSTECH and reviewed by both NAVER and Intel. This work was also supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government(MSIT) (No.RS-2023-00229849,MPU/Connectivity/TinyML SoC Solution for IoT Intelligence with Foundry-based eFLASH), and by National R&D Program through the National Research Foundation of Korea(NRF) funded by Ministry of Science and ICT(RS-2023-00258227).

REFERENCES

- [1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altschmidt, S. Altman, S. Anadkat *et al.*, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [2] G. Ali, S. Bhalachandra, N. Wright, A. Sill, and Y. Chen, “Evaluation of power controls and counters on general-purpose graphics processing units (gpus),” in *Poster: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020.
- [3] R. Anil, A. M. Dai, O. Firat, M. Johnson, D. Lepikhin, A. Passos, S. Shakeri, E. Taropa, P. Bailey, Z. Chen *et al.*, “PaLM 2 technical report,” *arXiv preprint arXiv:2305.10403*, 2023.
- [4] Y. Bai, Y.-X. Wang, and E. Liberty, “Proxquant: Quantized neural networks via proximal operators,” *arXiv preprint arXiv:1810.00861*, 2018.
- [5] R. Balasubramanian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, “CACTI 7: New tools for interconnect exploration in innovative off-chip memories,” *ACM Trans. Archit. Code Optim.*, vol. 14, no. 2, 2017.
- [6] N. Burgess, J. Milanovic, N. Stephens, K. Monachopoulos, and D. Mansell, “Bfloat16 processing for neural networks,” in *2019 IEEE 26th Symposium on Computer Arithmetic (ARITH)*. IEEE, 2019, pp. 88–91.
- [7] J. Chee, Y. Cai, V. Kuleshov, and C. M. De Sa, “Quip: 2-bit quantization of large language models with guarantees,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [8] I. Chung, B. Kim, Y. Choi, S. J. Kwon, Y. Jeon, B. Park, S. Kim, and D. Lee, “Extremely low bit transformer quantization for on-device neural machine translation,” *arXiv preprint arXiv:2009.07453*, 2020.
- [9] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “Gpt3. int8 (:): 8-bit matrix multiplication for transformers at scale,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 30 318–30 332, 2022.
- [10] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “OPTQ: Accurate quantization for generative pre-trained transformers,” in *The Eleventh International Conference on Learning Representations*, 2022.
- [11] H. Genc, S. Kim, A. Amid, A. Haj-Ali, V. Iyer, P. Prakash, J. Zhao, D. Grubb, H. Liew, H. Mao *et al.*, “Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration,” in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 769–774.
- [12] C. Gou and G. N. Gaydadjiev, “Addressing gpu on-chip shared memory bank conflicts using elastic pipeline,” *International Journal of Parallel Programming*, vol. 41, pp. 400–429, 2013.
- [13] G. Henry, P. T. P. Tang, and A. Heinecke, “Leveraging the bfloat16 artificial intelligence datatype for higher-precision computations,” in *2019 IEEE 26th Symposium on Computer Arithmetic (ARITH)*. IEEE, 2019, pp. 69–76.
- [14] J. H. Heo, J. Kim, B. Kwon, B. Kim, S. J. Kwon, and D. Lee, “Rethinking channel dimensions to isolate outliers for low-bit weight quantization of large language models,” *arXiv preprint arXiv:2309.15531*, 2023.
- [15] W. Huang, Y. Liu, H. Qin, Y. Li, S. Zhang, X. Liu, M. Magno, and X. Qi, “Billm: Pushing the limit of post-training quantization for llms,” *arXiv preprint arXiv:2402.04291*, 2024.
- [16] J. Jang, Y. Kim, J. Lee, and J.-J. Kim, “FIGNA: Integer unit-based accelerator design for fp-int gemm preserving numerical accuracy,” in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 760–773.
- [17] Y. Jeon, B. Park, S. J. Kwon, B. Kim, J. Yun, and D. Lee, “BiQGEMM: matrix multiplication with lookup table for binary-coding-based quantized dnns,” in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2020, pp. 1–14.
- [18] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers *et al.*, “In-datacenter performance analysis of a tensor processing unit,” in *Proceedings of the 44th annual international symposium on computer architecture*, 2017, pp. 1–12.
- [19] J. Kim, J. H. Lee, S. Kim, J. Park, K. M. Yoo, S. J. Kwon, and D. Lee, “Memory-efficient fine-tuning of compressed large language models via sub-4-bit integer quantization,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [20] S. Kim, C. Hooper, A. Gholami, Z. Dong, X. Li, S. Shen, M. W. Mahoney, and K. Keutzer, “Squeezellm: Dense-and-sparse quantization,” *arXiv preprint arXiv:2306.07629*, 2023.
- [21] Y. J. Kim, R. Henry, R. Fahim, and H. H. Awadalla, “Who says elephants can’t run: Bringing large scale moe models into cloud scale production,” *arXiv preprint arXiv:2211.10017*, 2022.
- [22] Y. Kim, J. Jang, J. Lee, J. Park, J. Kim, B. Kim, S. J. Kwon, D. Lee *et al.*, “Winning both the accuracy of floating point activation and the simplicity of integer arithmetic,” in *The Eleventh International Conference on Learning Representations*, 2023.
- [23] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 611–626.
- [24] J. H. Lee, J. Kim, S. J. Kwon, and D. Lee, “Flexround: Learnable rounding based on element-wise division for post-training quantization,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 18 913–18 939.
- [25] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “AWQ: Activation-aware weight quantization for on-device llm compression and acceleration,” *Proceedings of Machine Learning and Systems*, vol. 6, pp. 87–100, 2024.
- [26] A. S. Luccioni, S. Viguer, and A.-L. Ligozat, “Estimating the carbon footprint of bloom, a 176b parameter language model,” *Journal of Machine Learning Research*, vol. 24, no. 253, pp. 1–15, 2023.
- [27] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” *arXiv preprint arXiv:1609.07843*, 2016.
- [28] G. Park, B. park, M. Kim, S. Lee, J. Kim, B. Kwon, S. J. Kwon, B. Kim, Y. Lee, and D. Lee, “LUT-GEMM: Quantized matrix multiplication based on LUTs for efficient inference in large-scale generative language models,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [29] Y. Sun, L. Dong, S. Huang, S. Ma, Y. Xia, J. Xue, J. Wang, and F. Wei, “Retentive network: A successor to transformer for large language models,” *arXiv preprint arXiv:2307.08621*, 2023.
- [30] D. Tiwari, S. Gupta, G. Gallarno, J. Rogers, and D. Maxwell, “Reliability lessons learned from gpu experience with the titan supercomputer at oak ridge leadership computing facility,” in *Proceedings of the international conference for high performance computing, networking, storage and analysis*, 2015, pp. 1–12.
- [31] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [32] H. Wang, S. Ma, L. Dong, S. Huang, H. Wang, L. Ma, F. Yang, R. Wang, Y. Wu, and F. Wei, “Bitnet: Scaling 1-bit transformers for large language models,” *arXiv preprint arXiv:2310.11453*, 2023.
- [33] C. Xu, J. Yao, Z. Lin, W. Ou, Y. Cao, Z. Wang, and H. Zha, “Alternating multi-bit quantization for recurrent neural networks,” *arXiv preprint arXiv:1802.00150*, 2018.
- [34] Z. Yao, R. Yazdani Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, “Zeroquant: Efficient and affordable post-training quantization for large-scale transformers,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 168–27 183, 2022.
- [35] E. Yoo, G. Park, J. G. Min, S. J. Kwon, B. Park, D. Lee, and Y. Lee, “Tf-mvp: Novel sparsity-aware transformer accelerator with mixed-length vector pruning,” in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6.
- [36] H. You, Y. Guo, Y. Fu, W. Zhou, H. Shi, X. Zhang, S. Kundu, A. Yazdanbakhsh, and Y. Lin, “ShiftAddLLM: Accelerating pretrained llms via post-training multiplication-less reparameterization,” *arXiv preprint arXiv:2406.05981*, 2024.

- [37] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for {Transformer-Based} generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 521–538.
- [38] Z. Yuan, L. Niu, J. Liu, W. Liu, X. Wang, Y. Shang, G. Sun, Q. Wu, J. Wu, and B. Wu, "Rptq: Reorder-based post-training quantization for large language models," *arXiv preprint arXiv:2304.01089*, 2023.
- [39] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin *et al.*, "Opt: Open pre-trained transformer language models," *arXiv preprint arXiv:2205.01068*, 2022.